

IFRO Digital & Verification Summer School

General-Purpose Counter

Design Section

A general-purpose counter can be used in different modes and configurations, such as:

- Count different events inside a system,
- Count starting with a configurable event,
- Measure the distance between two events,
- Generate signals with a defined shape.

The 10-bit general-purpose counter that will be implemented will have the following functions:

- *PWM mode*: Pulse-width modulation (PWM) is any method of representing a signal as a rectangular wave with a varying duty cycle. PWM is useful for controlling the average power or amplitude delivered by an electrical signal. The average value of voltage (and current) fed to the load is controlled by switching the supply between 0 and 100% at a rate faster than it takes the load to change significantly. The current implementation will generate a PWM signal for which both frequency and duty cycle will vary.
- *Counter mode*: Counter mode will be used to count events on a selectable source until a maximum value is reached. This source can be the rising, falling, or both edges of an input source signal. The current value of the counter can be read through a status register. When the maximum value is reached (configurable), an output will be activated according to an out-function configuration.
- *Timer mode*: Timer mode will be used to count starting from a selectable event (rising, falling, or both edges) on a selectable input source signal until a maximum value is reached. From this start condition, the counting will be done using the clock signal. The current value of the counter can be read through a status register. When the maximum value is reached (configurable), an output will be activated according to an out-function configuration. If another starting condition is detected before the maximum value is reached, the counting will restart.
- *Capture*: Capture is just an extra option for the already existing counter and timer modes. With this function, an extra event - capture event - is defined and the value of the running counter is stored when this event is triggered. The output of the module is not affected by the capture logic.

The proposed architecture for the general-purpose counter is described in Figure 1.

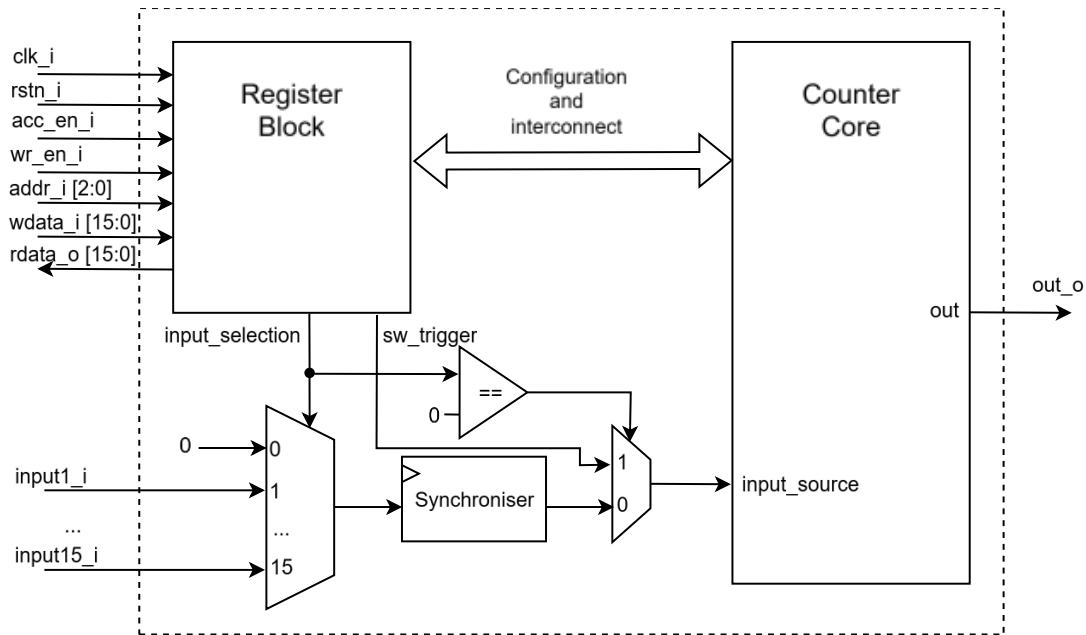


Figure 1 General-Purpose Counter Architecture

The architecture contains the following main components:

- A synchronous interface used to configure the counter by writing and reading the configuration and status registers,
- A register block which contains all the configuration and status fields,
- A Synchroniser (FF) for the asynchronous inputs (inputx_i, x=1...n),
- One single counter and the logic needed to implement all the different operational modes.

1 Module interface

Port name	Size	Type	Clock domain	Description
clk_i	1	input	n.a.	Clock signal with target frequency of 40Mhz
rstn_i	1	input	asynchronous	Asynchronous reset, active low
acc_en_i	1	input	clk_i	Access enable for synchronous transactions
wr_en_i	1	input	clk_i	Write enable. When set, a write is performed, when reset, a read is performed
addr_i	3	input	clk_i	Address accessed during a transaction
wdata_i	15	input	clk_i	Data to be written during write access

rdata_o	15	output	clk_i	Data read during read access
inputx_i	1	input	asynchronous	Inputs that can be selected as an event source for the general-purpose counter
out_o	1	output	clk_i	Output calculated based on operation mode and out function

2 Communication interface

A protocol is a set of rules that define how data is transmitted and received over a network or communication channel. Protocols define the format, timing, sequencing, and error control for data communication. They also specify the types of messages that can be exchanged between devices and how those messages are interpreted. Depending on the application for which they were defined, protocols are very different and can be grouped in different categories. In this application, a synchronous and parallel protocol will be used.

Synchronous communication requires that the clocks in the transmitting and receiving devices are synchronised – running at the same rate – so the receiver can sample the signal at the same time intervals used by the transmitter. Therefore, synchronous protocols always have an extra wire for the external clock signal.

In parallel communication, multiple bits of data are sent simultaneously, as opposed to serial communication, which involves sending data one bit at a time.

The communication interface is used to transfer data to and from the configuration and status registers. A certain register is identified using an address. Those registers can be read only (R), write only (W), or read and write (R/W). The timing for write and read commands is shown in Figure 2 and Figure 3.

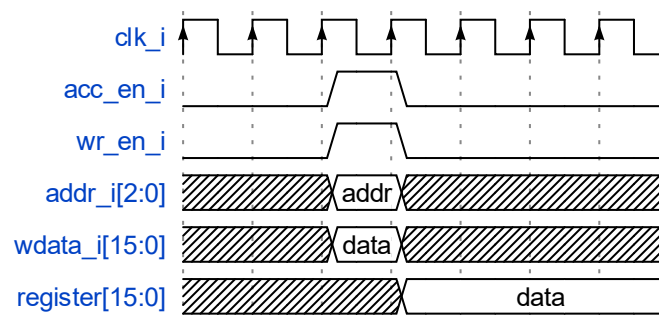


Figure 2 Write access – register is identified using addr

For write access, the address used to identify the register and the data to be written are available in the same clock cycle in which *acc_en_i* and *wr_en_i* are set. The written data will be available in the target register after the next positive edge event on the clock signal.

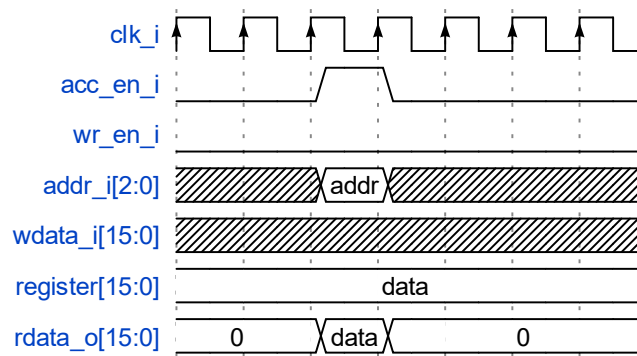


Figure 3 Read access – register is identified using `addr`

For read access, the address used to identify the register is available in the same clock cycle in which `acc_en_i` is set and `wr_en_i` is 0. The read data will be available on `rdata_o` on the same clock cycle in which `acc_en_i` is set (the read data process is asynchronous).

3 Register block

The register block contains the configuration and status bits of the counter and will facilitate the access to status information. Each register is identified by an address and is completely described in the following section.

CTRL0			Address: 0 Reset value: 16'd0
Bits	Bit field name	Access	Description
[1:0]	Mode	R/W	These bits are used to enable the counter and set the operation mode: 2'b00 – module disabled 2'b01 – module in PWM mode 2'b10 – module in counter mode 2'b11 – module in timer mode
[15:2]	n.a.	n.a.	Not used

PWM_MODE			Address: 1 Reset value: 16'd0
Bits	Bit field name	Access	Description
[9:0]	Duty cycle	R/W	These bits define the duty cycle of the PWM signal as follows: 10'd0 – 0% duty cycle ... 10'd999 – 100% duty cycle 10'd1000 ... 10'd1023 – 100% duty cycle
[11:10]	n.a.	n.a.	Not used
[13:12]	Frequency selection	R/W	These bits define the frequency of the PWM signal as follows: 2'b00 – 100 Hz 2'b01 – 200 Hz 2'b10 – 320 Hz 2'b11 – 400 Hz
[15:14]	n.a.	n.a.	Not used

CNT_MODE0			Address: 2 Reset value: 16'd0
Bits	Bit field name	Access	Description
[3:0]	Input selection	R/W	These bits are used to select one of the 16 inputs as follows: 4'b0000 – software input 4'b0001 – input1 ... 4'b1111 – input15
[5:4]	Trigger selection	R/W	These bits are used to select the trigger option for the counter/time mode as follows: 2'b00 – rising edge used as trigger 2'b01 – falling edge used as trigger 2'b10 – both edges used as trigger 2'b11 – none (not used)
[7:6]	n.a.	n.a.	Not used
[8]	Out function	R/W	This bit defines the format of the output signal as follows: 1'b0 – output signal is active only one clock cycle (pulse generator) 1'b1 – output signal toggles when the according condition is met
[11:9]	n.a.	n.a.	Not used
[13:12]	Capture selection	R/W	These bits are used to select the capture event as follows: 2'b00 – rising edge used as trigger 2'b01 – falling edge used as trigger 2'b10 – both edges used as trigger 2'b11 – none (not used)
[15:14]	n.a.	n.a.	Not used

CNT_MODE1			Address: 3 Reset value: 16'd0
Bits	Bit field name	Access	Description
[9:0]	Target value	R/W	These bits are used to set the target value for the counter. When this value is reached, the output will be activated according to the selected output function. The reset value for this is 0. Once the target value is reached, the internal counter will be reset automatically.
[15:10]	n.a.	n.a.	Not used

COUNTER_VALUE			Address: 4 Reset value: n.a.
Bits	Bit field name	Access	Description
[9:0]	Counter value	R	These bits reflect the current value of the internal counter.
[15:10]	n.a.	n.a.	Not used

COMMAND			Address: 5 Reset value: n.a.
Bits	Bit field name	Access	Description
[0]	Clear	W	This bit will clear the internal counter.
[3:1]	n.a.	n.a.	Not used
[4]	SW Trigger	W	This bit is used as software input and will be used by the internal counter if the selection input is set to 0.
[15:5]	n.a.	n.a.	Not used

The main components are the Main Counter (in this case, the 10-bit counter), the frequency control stages (one or more, depending on the target accuracy for the generated signal frequency), and the duty cycle control stage.

In the proposed architecture, the first frequency control stage will generate at its output (*prescaler*) a one clock cycle pulse at each $limit_f1 + 1$ clock cycles. Being connected to the Main Counter enable signal, this will lead to the incrementation of the Main Counter's value at each $limit_f1 + 1$ clock cycles, having the same effect as dividing the frequency of the Main Counter by $limit_f1 + 1$.

The typical architecture of such a module and the resulting *prescaler* are presented in Figure 5 and 6.

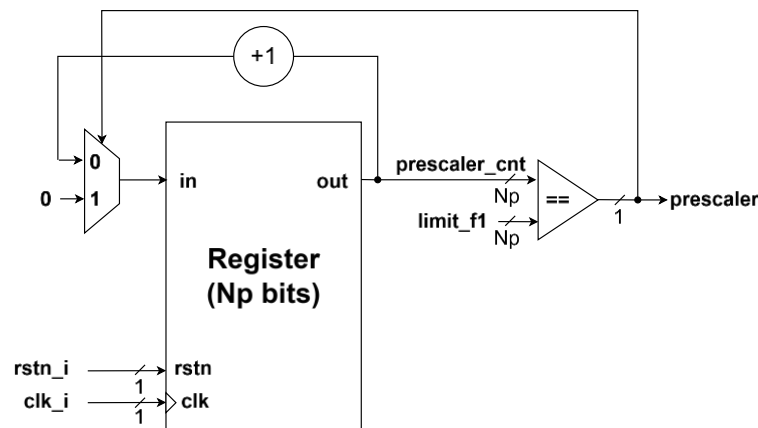


Figure 5 PWM frequency control architecture

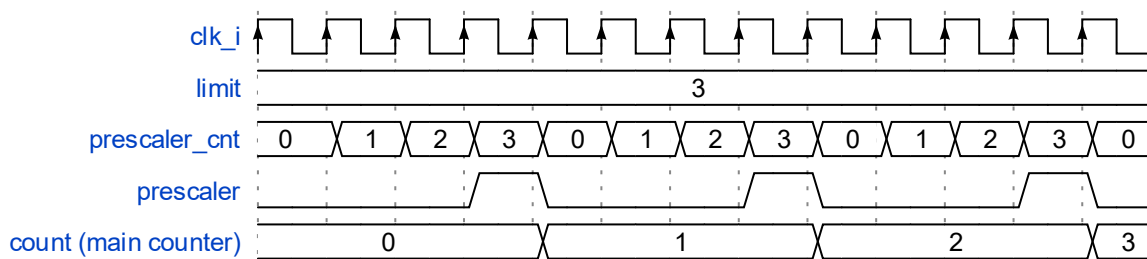


Figure 6 Frequency control example (frequency division by 4 ($limit + 1$))

The second frequency control stage (the one over the Main Counter) can influence the pwm signal frequency by limiting the maximum value of the counter to $limit_f2$.

Considering the two frequency control stages, the pwm signal period will be equal to $(limit_f2 + 1) * (limit_f1 + 1) * T_{clk_i}$. In this way, choosing the right values for $limit_f1$ and $limit_f2$ the target values for the pwm signal frequencies can be achieved. In the current design, to make the implementation of the circuit easier, $limit_f2 = 999$ will be used.

The duty cycle is usually controlled by a comparison module. The pwm signals with different duty cycles for a 2-bit counter are presented in Figure 7.

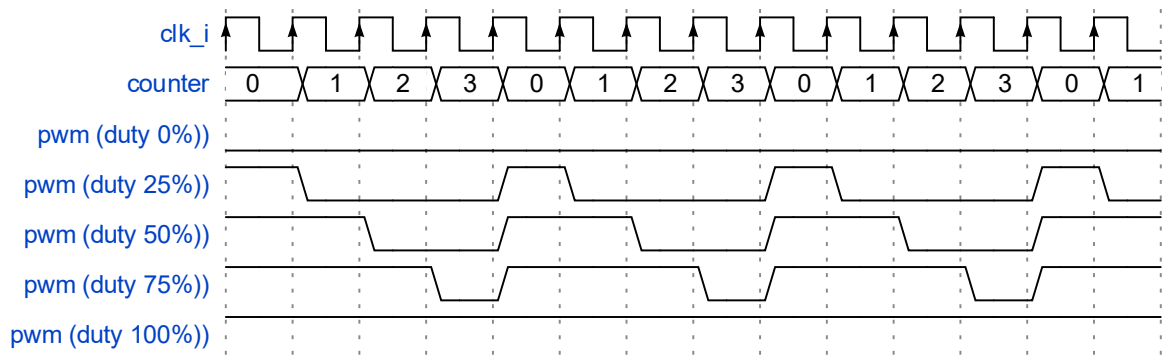


Figure 7 PWM signals for a 2-bit counter

4.2 Counter mode

Counter mode will be used to count events on a selectable source. The current value of the counter can be read through a status register. When the maximum value is reached (configurable), the output will be activated according to the out-function bit.

Therefore, in Counter mode, the main counter will behave as follows:

- It increases its value when the selected event (rising, falling, or both edges) occurs on the selected input.
- If the target value is reached, it will be reloaded with 0 on the next positive edge on the clock.

The typical architecture for edge event detection is presented in Figure 8 and an example of generated signals is presented in Figure 9.

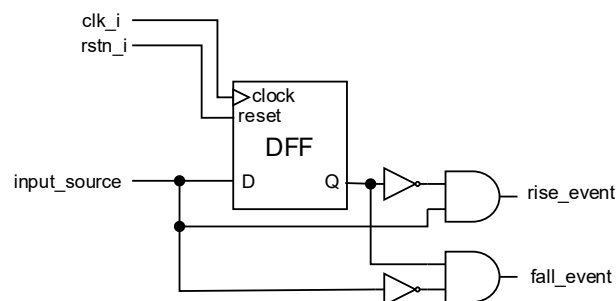


Figure 8 Edge events detection circuit

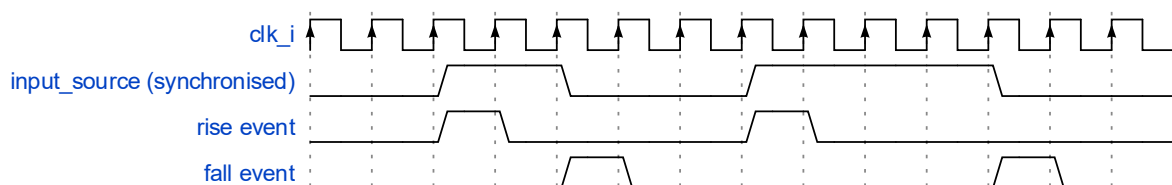


Figure 9 Edge events example

In the next Figure, the counter behavior in Counter mode with trigger selection (CNT_MODE0[5:4]) equal to 0 (rising edge), input selection selecting one of the 16 asynchronous inputs, and target value equal to 3 is presented.

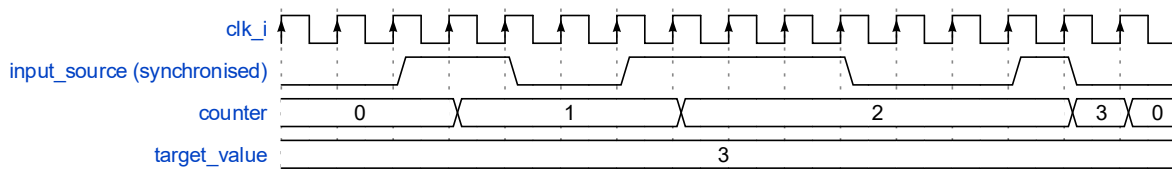


Figure 10 Counter mode waveform

4.3 Timer mode

Timer mode will be used to count starting from a selectable event (rising, falling, or both edges) on a selectable input source signal until a maximum value is reached. From this start condition, the counting will be done using the clock signal. If another starting condition is detected before the maximum value is reached, the counting will restart.

Therefore, in Timer mode, the main counter will behave as follows:

- It increases its value starting with the selected event (rising, falling, or both edges) occurrence on the selected input until it reaches the target value.
- If the target value is reached, it will be reloaded with 0 on the next positive edge on the clock, waiting for the next event to start counting again.
- If another selected event occurs on the selected input, the counter will start to count again (next value after the event will be 1).

In the next Figure, the counter behavior in Timer mode with trigger selection (CNT_MODE0[5:4]) equal to 0 (rising edge), input selection selecting one of the 16 asynchronous inputs, and target value equal to 3 is presented.

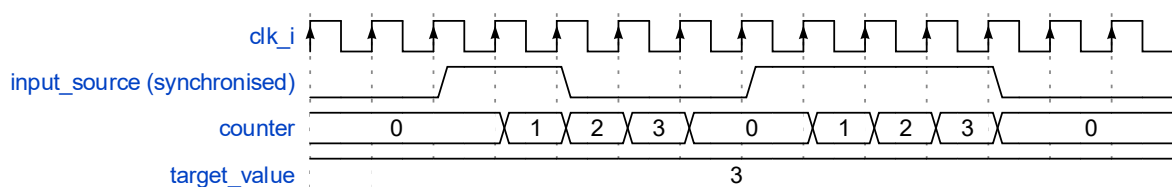


Figure 11 Timer mode waveform

5 Capture feature

Capture is just an extra option for the already existing counter and timer modes. With this function, an extra event - capture event - is defined and the value of the running counter is stored in a register when this event is triggered. The output of the module is not affected by the capture logic.

In the next Figure, the capture behavior in Timer mode with trigger selection (CNT_MODE0[5:4]) equal to 0 (rising edge), input selection selecting one of the 16 asynchronous inputs, and target value equal to 3 is presented. The capture event trigger (CNT_MODE0[13:12]) is defined as the falling edge of the input source signal (the synchronised selected inputx).

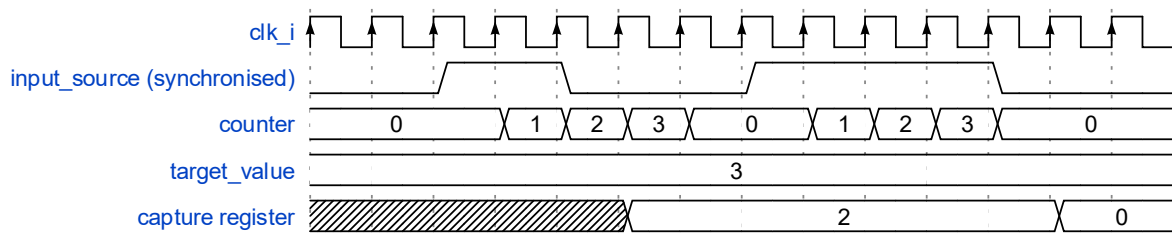


Figure 12 Capture example

6 Output signal

The output signal is a synchronous signal (synchronised using a FF) and depends on the operation mode and the out-function bit in CNT_MODE0.

Therefore, the out signal will behave as follows:

- If operation mode is PWM: out is the synchronised PWM signal. If additional restrictions are applied (e.g., in the current design the duty cycle will be 100% for PWM_MODE[9:0] $\geq$ 999), they can be implemented here.
- If operation mode is Counter or Timer:
 - If the out-function bit is 0: pulse mode - out is set for only one clock cycle, starting on the next positive edge after the counter reaches the target value.
 - If the out-function bit is 1: toggle mode - out toggles (complement its value) on the next positive edge after the counter reaches the target value.

In the next Figures, the out signal behaviour is presented for Timer mode (trigger selection (CNT_MODE0[5:4]) equal to 0 (rising edge), input selection selecting one of the 16 asynchronous inputs, and target value equal to 3) for both out-function equal to 0 (pulse mode) and 1 (toggle mode).

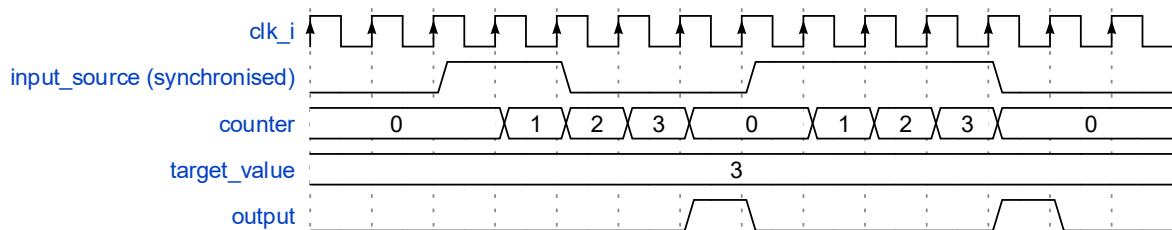


Figure 13 Output in pulse mode

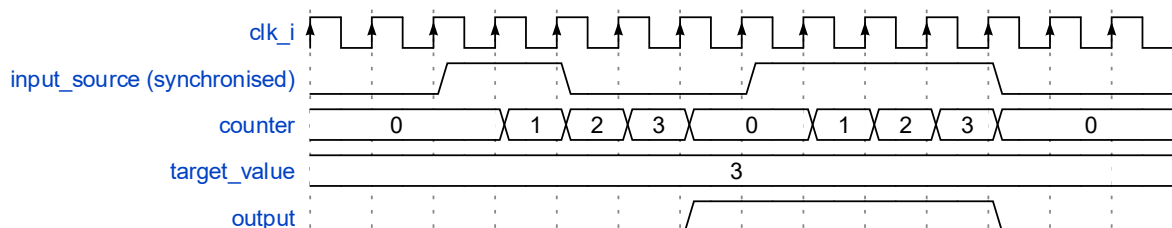


Figure 14 Output in toggle mode

7 Task scheduler

Day 1:

- *Digital Design Introduction, HW vs SW, programming languages vs hardware description languages (mentors)*
- *Synchronous parallel protocol explanations & discussions (all)*
- *Interface + register block implementation (students)*

Day 2:

- *Testbench for existing blocks (students)*
- *PWM mode explanations & discussions (all)*
- *Counter for PWM mode implementation (students)*

Day 3:

- *Testbench for existing blocks(students)*
- *Counter mode & timer mode explanations & discussions (all)*
- *Counter mode implementation (students)*
- *Timer mode implementation(students)*

Day 4:

- *Top level instantiations & tests for timer/counter mode(students)*
- *Digital Design Backend, RTL -> chip (mentors)*